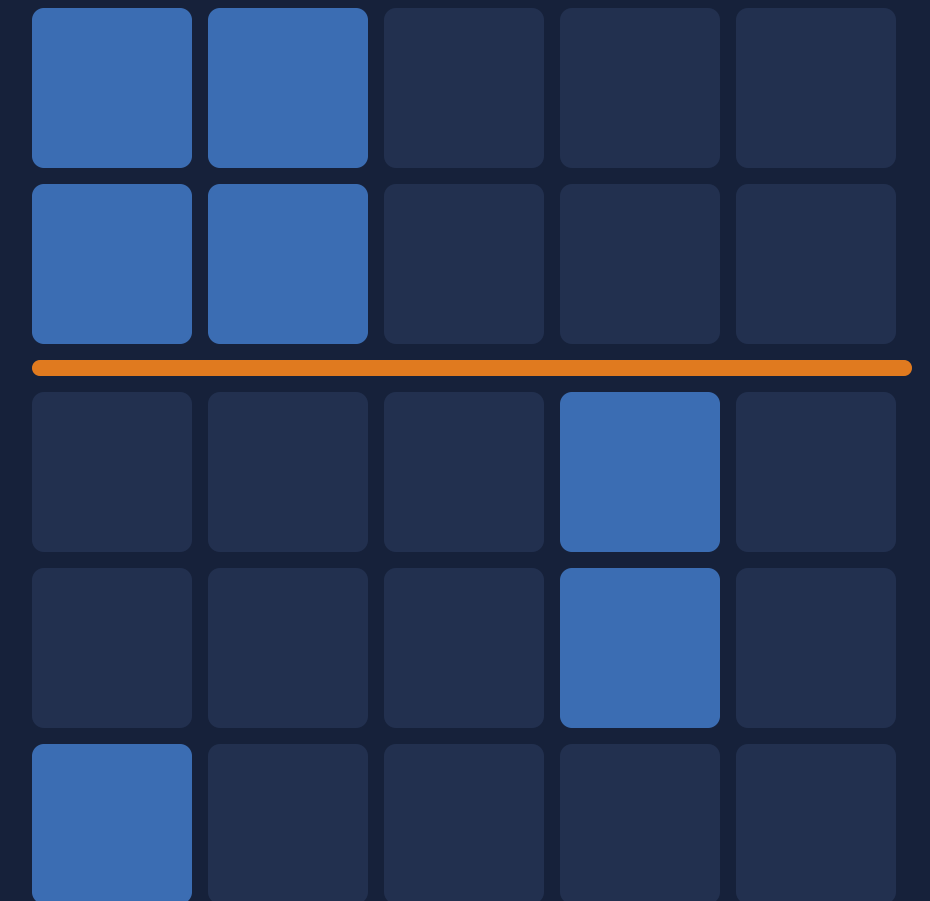


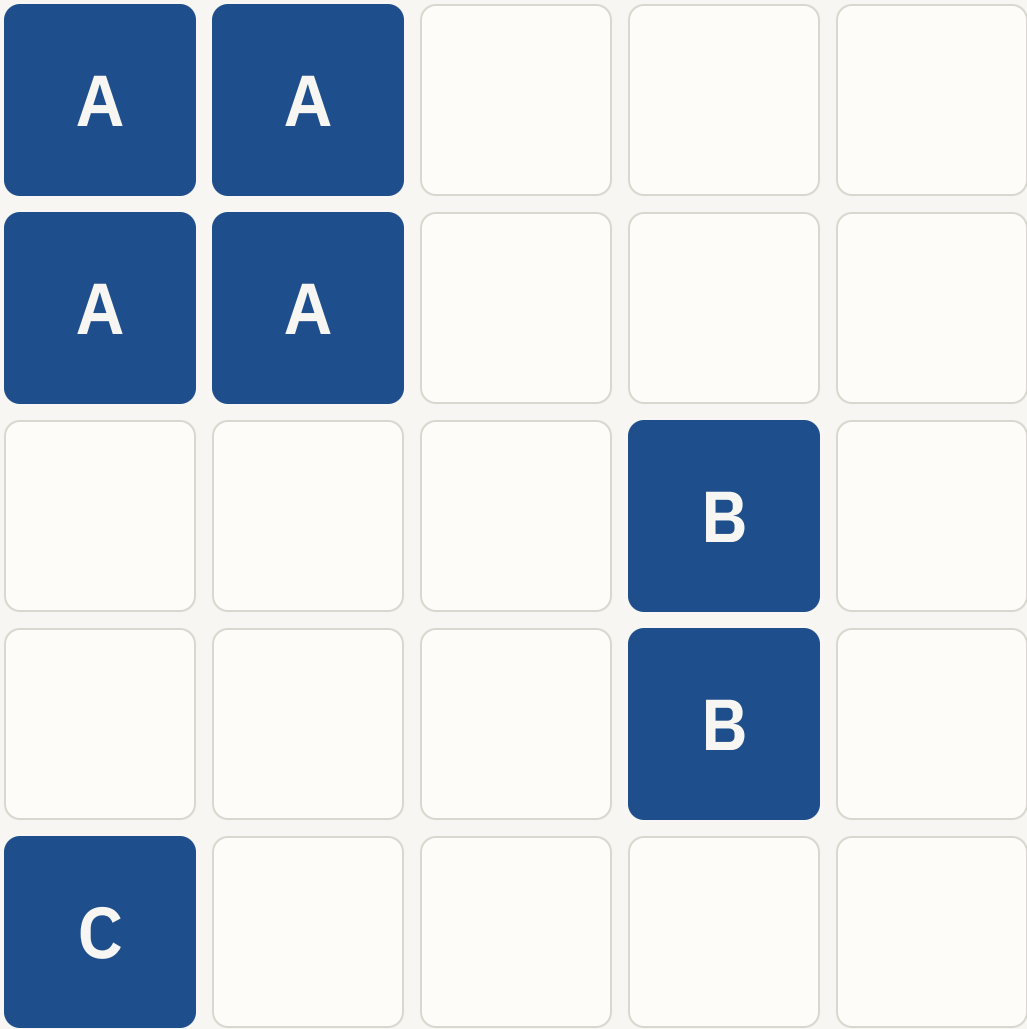
Contagem paralela de objetos em uma matriz binária

Versão sequencial e versão paralela com Pthreads, em ANSI C (C89)

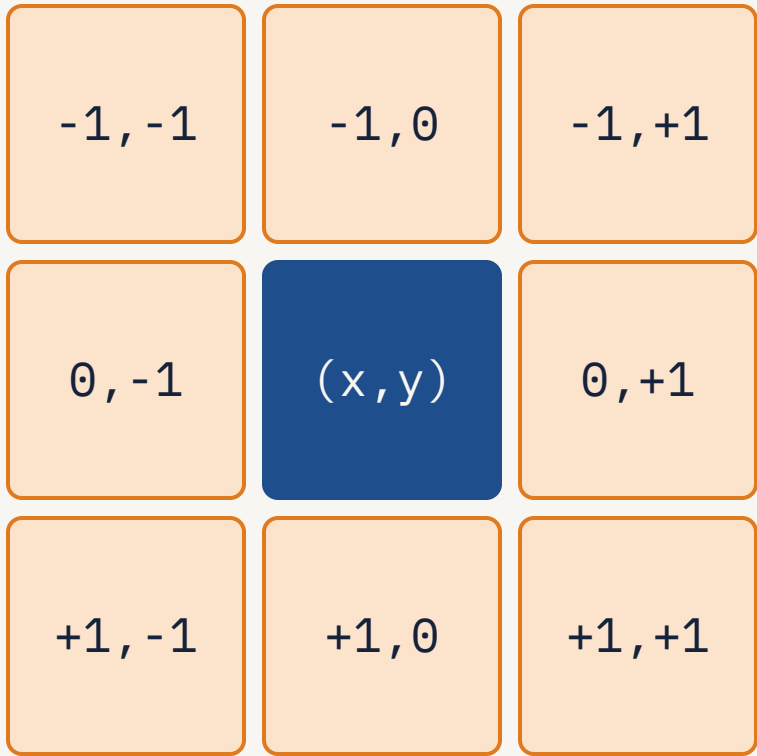


O PROBLEMA

Contar objetos com conectividade 8



Exemplo 1 (5 × 5): 3 objetos



Os 8 vizinhos de uma célula

Objeto = grupo de células 1 ligadas por um lado **ou por um canto**.

Os deslocamentos ao lado são os vetores **direction_x** e **direction_y** do código.

Entrada: arquivo texto com “LINHAS COLUNAS” na primeira linha e depois a matriz de 0 e 1.

Dividir, rotular e costurar as fronteiras

1

Dividir

A matriz é cortada em faixas de linhas, uma faixa por thread.

2

Rotular

Em paralelo, cada thread faz flood fill só na sua faixa e grava um rótulo em cada célula.

3

Costurar

Após o `pthread_join`, a thread principal une os rótulos que se tocam na fronteira.

`objetos =  $\Sigma$  objetos locais – uniões na fronteira`

VERSÃO SEQUENCIAL

Varredura + flood fill iterativo

```
/* countObjects: varre a matriz */
for (i = 0; i < ROW; i++)
    for (j = 0; j < COL; j++)
        if (mat[i][j] == 1 && !visited[i][j]) {
            count++;
            floodFill(i, j, mat, visited, pilha);
        }

/* floodFill: pilha explícita */
while (topo > 0) {
    celula = pilha[--topo];
    for (i = 0; i < 8; i++) /* 8 vizinhos */
        if (vizinho == 1 && !visitado) {
            marca; empilha;
        }
}
```

Por que iterativo?

A versão recursiva original dava **segmentation fault** em 2000×2000 com 60% de 1s: um objeto gigante gera milhões de chamadas aninhadas e estoura a pilha.

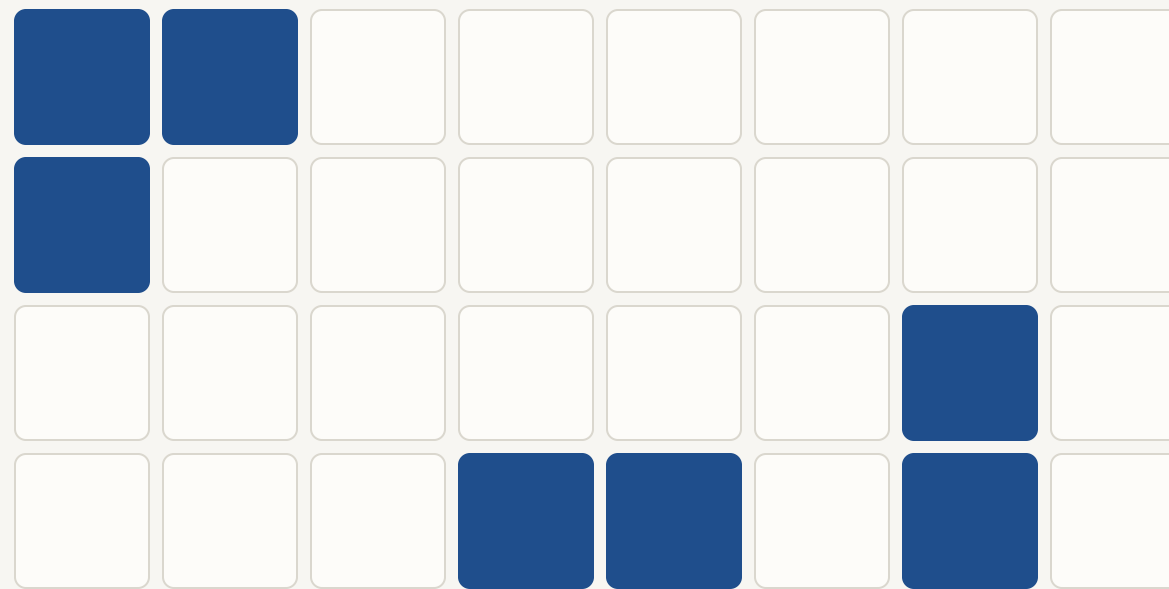
Papel no trabalho

Referência de correção para o paralelo e base de tempo do speedup.

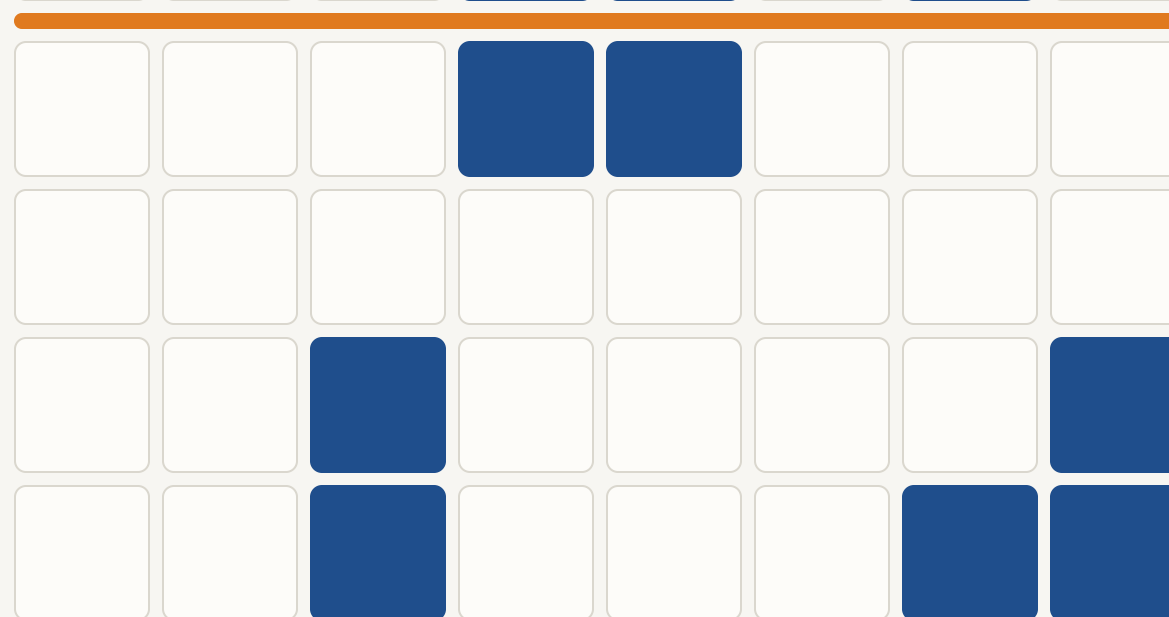
DECOMPOSIÇÃO

Uma faixa de linhas para cada thread

thread 0
linhas 0–3



thread 1
linhas 4–7



thread $t \rightarrow$ linhas $[t \cdot L/T, (t+1) \cdot L/T)$

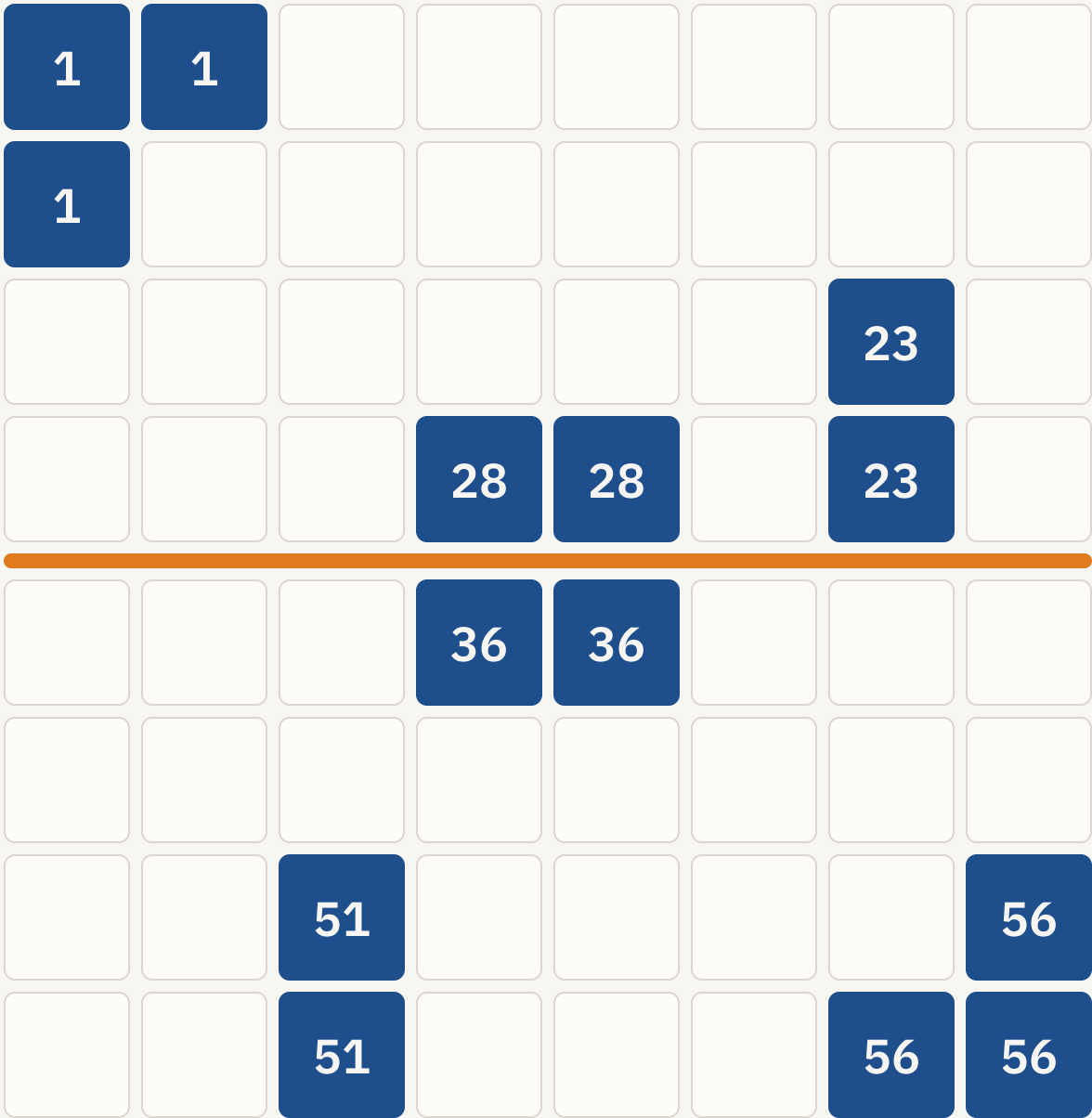
O número de threads vem da linha de comando: `./paralelo
arquivo.txt 4`

Cada linha fica inteira em uma thread, então as fronteiras
são **só horizontais**.

Com 2 threads (exemplos 1–3) e 3 threads (exemplos 4–5),
as fronteiras coincidem com as linhas laranja horizontais do
enunciado.

FASE PARALELA

Cada thread rotula os objetos da sua faixa



$$\text{rótulo} = \text{linha} \times \text{COL} + \text{coluna} + 1$$

Calculado na célula onde o objeto começou. É **único na matriz inteira** sem nenhum contador compartilhado.

Thread 0: 3 objetos locais (1, 23, 28)

Thread 1: 3 objetos locais (36, 51, 56)

O objeto 28 e o 36 são o mesmo objeto, cortado pela fronteira. Somar dá 6; o certo é 5.

Quem escreve o quê, e quando

Dado	Quem escreve	Por que é seguro
matrix	ninguém: só leitura	leituras simultâneas não conflitam
visited	dona da faixa, fase paralela	cada thread só toca as próprias linhas
dados_threads[t]	a thread t, ao terminar	uma posição por thread
pai[], total_duplicates	a principal, após o join	pthread_join é a barreira

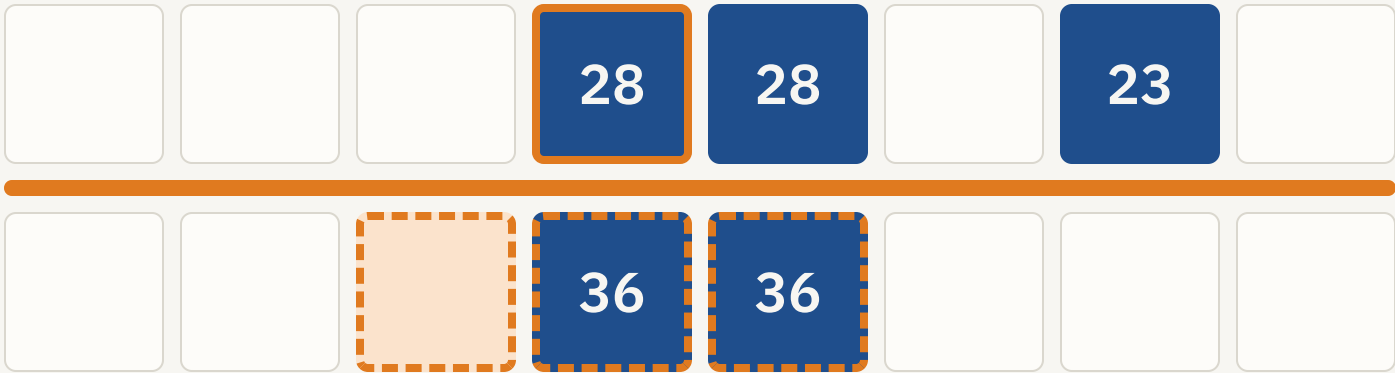
Nenhuma posição é escrita por duas threads: sem mutex, sem corrida, sem atualização perdida e sem deadlock.

Conferido com ThreadSanitizer e Helgrind (0 alertas) e Valgrind (0 vazamentos).

CONSOLIDAÇÃO

Costurando a fronteira com union-find

linha 3 · última da faixa 0



linha 4 · primeira da faixa 1

Para cada célula rotulada da linha de cima, o código olha os **3 vizinhos de baixo**: diagonal esquerda, vertical e diagonal direita ($d = -1, 0, +1$).

28 com 36: conjuntos diferentes → unir, 1 união

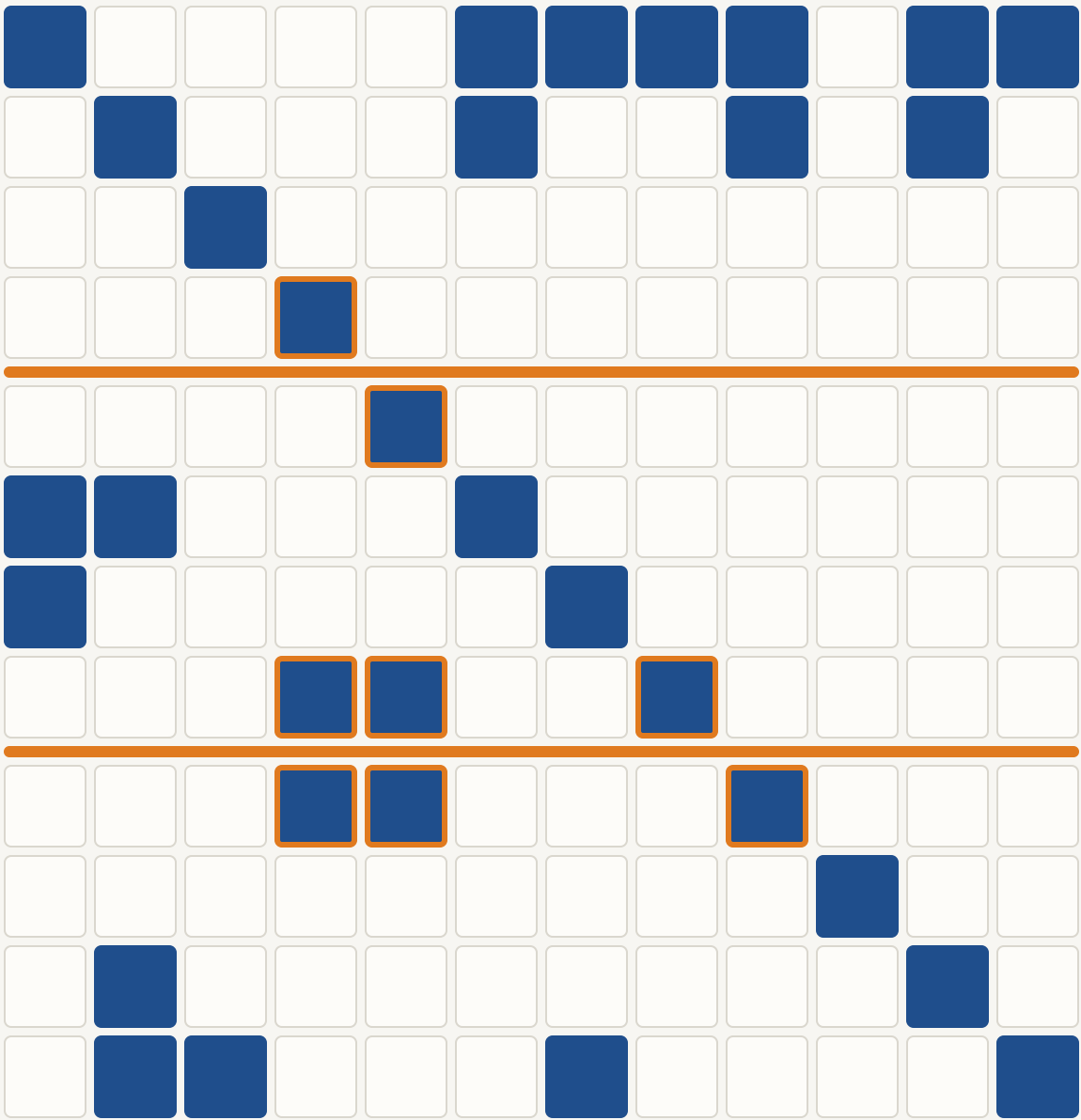
28 com 36 de novo: já unidos → não conta

23: nada embaixo → nada a fazer

$$3 + 3 - 1 = 5 \text{ objetos}$$

Por que union-find e não só contar pares? Um objeto pode tocar a fronteira várias vezes, ou juntar dois pedaços de outra faixa (forma de U). O union-find só conta uniões entre conjuntos ainda separados.

Exemplo 5: toque só pelo canto



3 threads: linhas 0–3, 4–7 e 8–11

Objetos locais: $3 + 3 + 4 = 10$

Unões (células com borda laranja): a diagonal longa cruza as duas fronteiras **só pelo canto**, (3,3)–(4,4) e (7,7)–(8,8); o quadrado cruza na vertical, (7,3)–(8,3).

$10 - 3 = 7$ objetos

Com faixas não há “encontro de 4 blocos”: o caso equivalente é o toque só pelo canto, coberto pelos vizinhos $d = \pm 1$.

DEMONSTRAÇÃO

No terminal

```
$ make
```

compila em C89 com -Wall -Wextra -pedantic

```
$ make test
```

5 obrigatórias + 200 matrizes aleatórias

```
$ ./paralelo tests/caso8_8.txt 2
```

objetos locais e uniões na fronteira

```
$ ./paralelo tests/caso12_12.txt 3
```

diagonais atravessando faixas

```
$ ./gera-matriz 4000 4000 60 42 > grande.txt
```

matriz grande (semente fixa)

```
$ ./sequencial grande.txt
```

tempo de referência

```
$ ./paralelo grande.txt 2
```

mesmo resultado, cerca de metade do tempo

Mesmo resultado em todas as matrizes

Ex.	Dimensões	Esperado	Sequencial	Paralelo (1–4 threads)	Status
1	5 × 5	3	3	3 3 3 3	OK
2	6 × 8	4	4	4 4 4 4	OK
3	8 × 8	5	5	5 5 5 5	OK
4	9 × 12	6	6	6 6 6 6	OK
5	12 × 12	7	7	7 7 7 7	OK

800 de 800

comparações iguais ao sequencial: 200 matrizes aleatórias ×
2, 3, 4 e 7 threads

0 erros

Valgrind: nenhum acesso inválido e nenhum vazamento de
memória

Como medimos

Máquina

VM Linux com 2 núcleos (Intel Xeon 2,1 GHz), 7 GB de RAM, gcc 13 com -O2.

Matrizes

1000², 2000², 4000² e 8000², com 30% e 60% de células 1, geradas com semente fixa.

Repetições

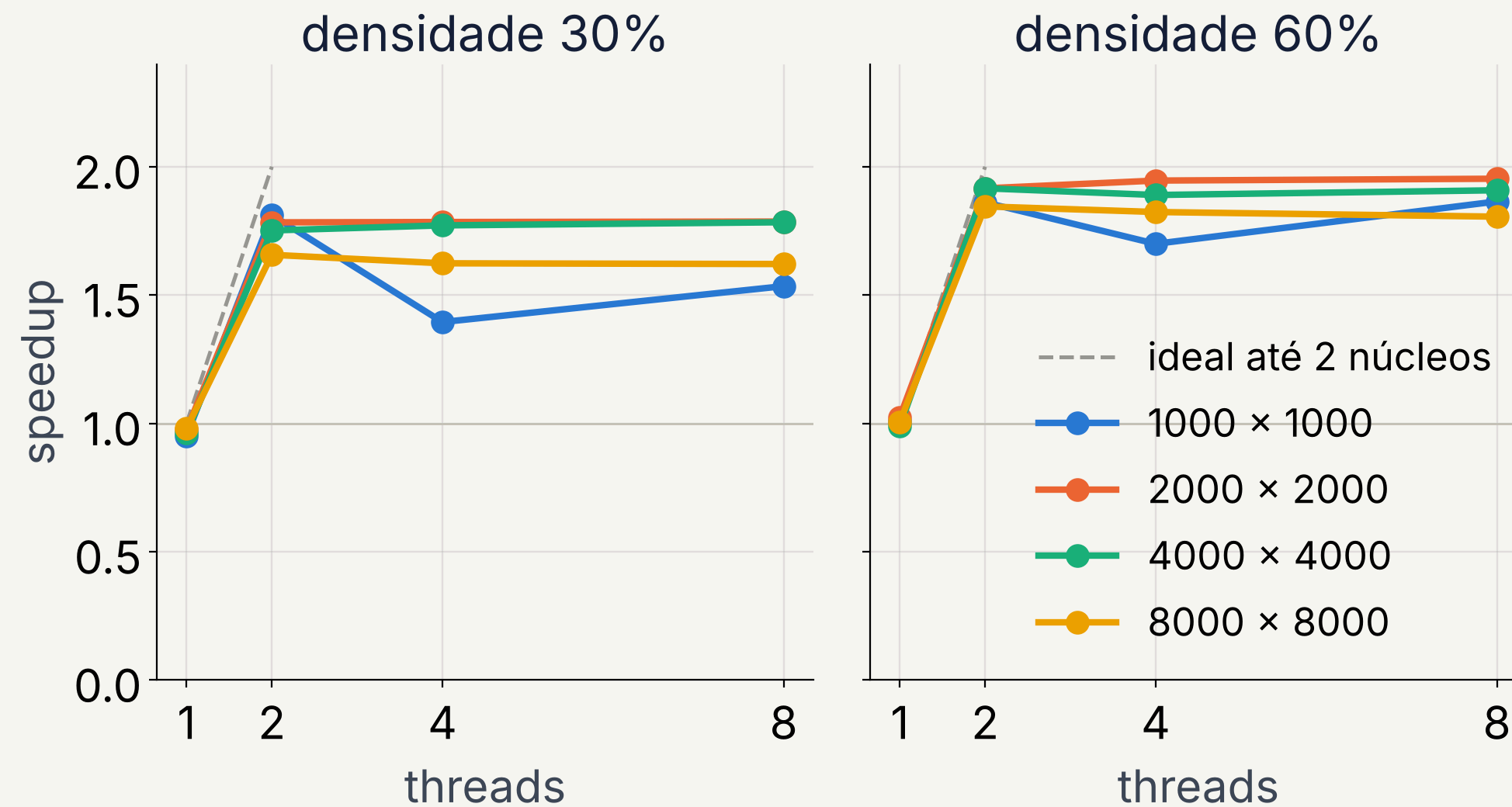
10 execuções por configuração, sequencial e paralelo intercalados. Valor usado: a mediana.

O que entra no tempo

Só a contagem (sem ler o arquivo). No paralelo: criar threads, join e consolidação.

RESULTADOS

$$\text{Speedup} = T_{\text{sequencial}} \div T_{\text{paralelo}}$$



1,7–1,9x

com 2 threads nas matrizes grandes (2 núcleos)

4 e 8 threads

sem ganho extra: só há 2 núcleos

1 thread ≈ 1x

0,95 a 1,02: custo extra do paralelo é pequeno

Quando o paralelo não compensa

Matrizes pequenas

4 μ s $\times$ 92 μ s

Exemplo 5: sequencial contra paralelo com 2 threads. Criar e esperar threads custa mais que contar 144 células, então nas 5 matrizes obrigatórias o paralelo é sempre mais lento.

Threads demais

1,81 $\times$ $\rightarrow$ 1,39 $\times$

1000², 30%: de 2 para 4 threads. Com 2 núcleos, as threads extras só se revezam no mesmo processador e pagam trocas de contexto.

Parte sequencial

35 ms $\rightarrow$ < 1 ms

Inicializar o vetor pai com um laço era ~15% do tempo com 2 threads (lei de Amdahl). Com calloc e “0 = raiz”, o custo sumiu.

CONCLUSÕES

O que o trabalho mostrou

Correto

5 de 5 matrizes obrigatórias e 800 comparações aleatórias iguais ao sequencial.

Paralelo de verdade

1,7× a 1,9× com 2 threads em 2 núcleos, de 1000^2 a 8000^2 .

Sincronização mínima

Só `pthread_join`: nenhuma escrita compartilhada, nenhum mutex, nenhum deadlock.

Próximo passo

Faixas fixas dependem de objetos bem espalhados; uma fila dinâmica de faixas equilibraria matrizes irregulares.